

1. INTRODUCTION

Content addressable memory (CAM) has been widely used in network devices for fast lookup functions. CAM implementation based on field programmable array (FPGA) has become a popular solution due to its flexibility. With the increasing capacity of CAM, reducing power consumption has been the main challenge for implementation on FPGA. This report investigates and implements low power schemes, pipelining and precomputation for RAM based CAM. The pipelining scheme divides RAM into several segments as pipeline. Mismatched RAM blocks disable the subsequent search operation in the following segments and therefore power consumption is reduced. For precomputation scheme, extra information is extracted from CAM words and input keys before the search operation. It saves power by filtering out mismatched blocks in the preconsumption stage.

In this work a complete power analysis of RAM-based CAM using xilinx will be performed. The comparison of the power consumption between the conventional scheme and low power scheme will be illustrated. Under the same test case, the average dynamic power consumption of CAM with pipelining scheme can be reduced by 86% compared to the conventional scheme. The precomputation scheme based on pipelining scheme further optimizes the power consumption of CAM. It decreases the final result of power estimation by 36% compared pipelining scheme.

2. CONTENT ADDRESSABLE MEMORY

2.1 INTRODUCTION

Content-addressable memory (CAM) is a special type of [computer memory](#) used in certain very-high-speed searching applications. It is also known as **associative memory** or **associative storage**^[a] and compares input search data (tag) against a table of stored data, and returns the address of matching data (or in the case of associative memory, the matching data).^[2]

CAM is frequently used in [networking devices](#) where it speeds [forwarding information base](#) and [routing table](#) operations. Several custom computers, like the Goodyear **STARAN**, were built to implement CAM.

2.2 HARDWARE ASSOCIATIVE ARRAY

Unlike standard computer memory ([random access memory](#) or RAM) in which the user supplies a memory address and the RAM returns the data word stored at that address, a CAM is designed such that the user supplies a data word and the CAM searches its entire memory to see if that data word is stored anywhere in it. If the data word is found, the CAM returns a list of one or more storage addresses where the word was found (and in some architectures, it also returns the

contents of that storage address, or other associated pieces of data). Thus, a CAM is the hardware embodiment of what in software terms would be called an [associative array](#). The data word recognition unit was proposed by [Dudley Allen Buck](#) in 1955.

2.3 STANDARDS FOR CONTENT ADDRESSABLE MEMORY

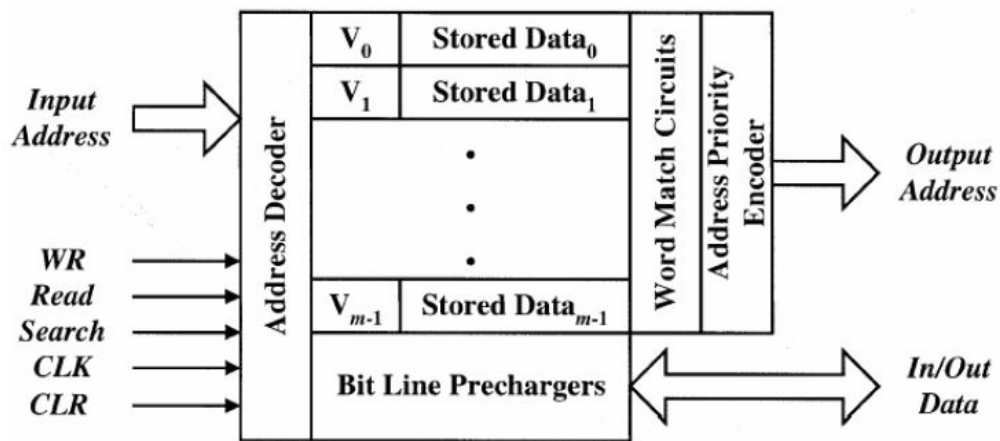
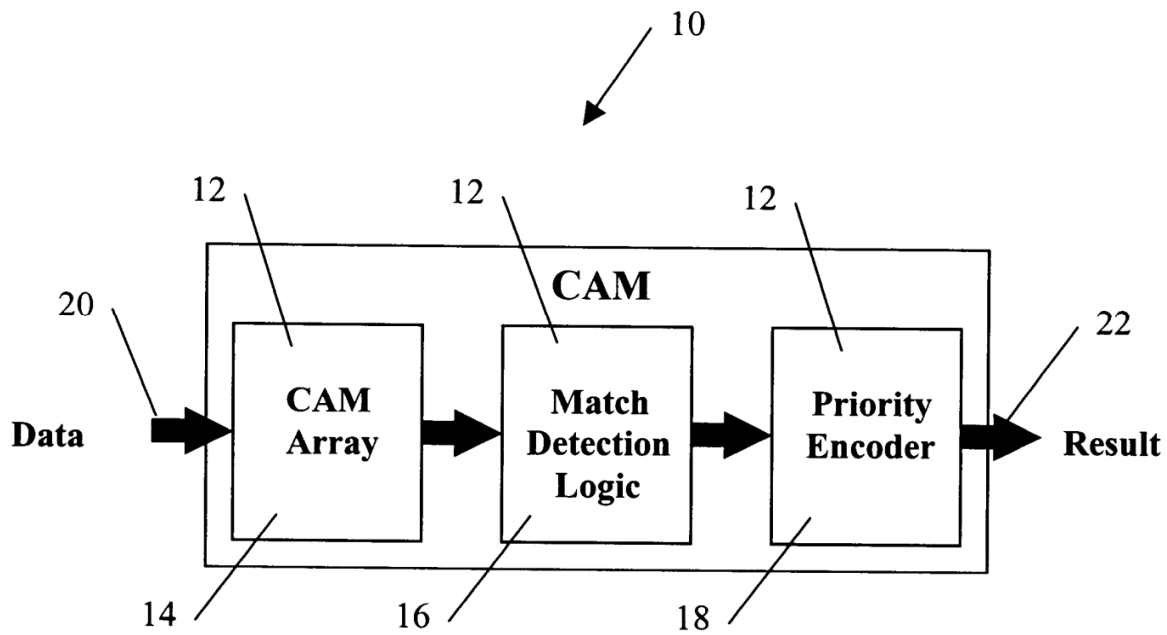
A major interface definition for CAMs and other [network search engines](#) (NSEs) was specified in an interoperability agreement called the [Look-Aside Interface](#) (LA-1 and LA-1B) developed by the [Network Processing Forum](#), which later merged with the [Optical Internetworking Forum](#) (OIF). Numerous devices have been produced by [Integrated Device Technology](#), [Cypress Semiconductor](#), [IBM](#), [Broadcom](#) and others to the LA interface agreement. On December 11, 2007, the OIF published the serial look a side (SLA) interface agreement.

2.3 SEMICONDUCTOR IMPLEMENTATION

Because a CAM is designed to search its entire memory in a single operation, it is much faster than RAM in virtually all search applications. There are cost disadvantages to CAM however. Unlike a RAM [chip](#), which has simple storage cells, each individual memory [bit](#) in a fully parallel CAM must have its own associated comparison circuit to detect a match between the stored bit and the input bit. Additionally, match outputs from each cell in the data word must be combined to yield a complete data word match signal. The additional circuitry increases the physical size and manufacturing cost of the CAM chip. The extra circuitry also increases power dissipation since every comparison circuit is active on every clock cycle. Consequently, CAM is only used in specialized applications where searching speed cannot be accomplished using a less costly method. One successful early implementation was a General Purpose Associative Processor IC and System.

2.4 APPLICATIONS

Because a CAM is designed to search its entire memory in a single operation, it is much faster than RAM in virtually all search applications. There are cost disadvantages to CAM however. Unlike a RAM [chip](#), which has simple storage cells, each individual memory [bit](#) in a fully parallel CAM must have its own associated comparison circuit to detect a match between the stored bit and the input bit. Additionally, match outputs from each cell in the data word must be combined to yield a complete data word match signal. The additional circuitry increases the physical size and manufacturing cost of the CAM chip. The extra circuitry also increases power dissipation since every comparison circuit is active on every clock cycle. Consequently, CAM is only used in specialized applications where searching speed cannot be accomplished using a less costly method. One successful early implementation was a General Purpose Associative Processor IC and System.



A general CAM architecture usually consists of data memory with a valid bit field, address decoder, word match circuit and address priority encoder, as shown in Figure 3.1. The memory organization of a traditional CAM consists of the data memory and a valid bit field, where it indicates the availability of stored data. In the data searching operation, the input data is sent into CAM to compare with all valid data stored in CAM simultaneously, and an address from those matches of comparison is sent to the output. In this architecture, CAM circuit performs large amount of comparison operations to identify all valid data stored in CAM during each data search operation. This comparison consumes most of the total CAM power. To minimize power consumption during the comparison, reducing the number of comparison operations is one of the best approaches. In order to reduce the number of bit

comparisons, pre computation techniques are involved. In the past decade, more research on energy reduction has focused on the circuit and technology domains.

3. DUAL PORT SRAM

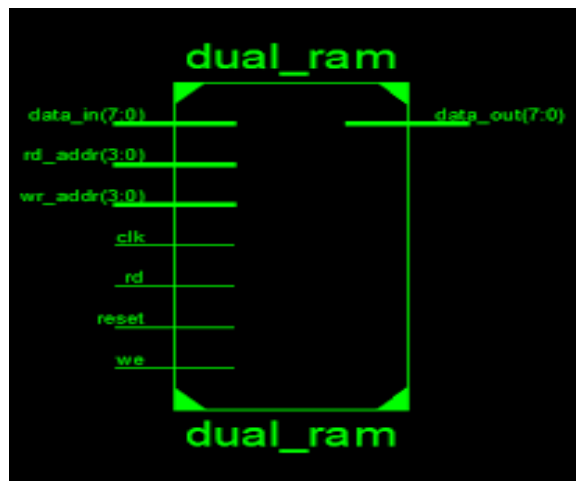
3.1 INTRODUCTION

Dual-ported RAM (DPRAM) is a type of [random-access memory](#) that allows multiple reads or writes to occur at the same time, or nearly the same time, unlike single-ported RAM which allows only one access at a time.

Video RAM or [VRAM](#) is a common form of dual-ported [dynamic RAM](#) mostly used for video memory, allowing the [CPU](#) to draw the image at the same time the video hardware is reading it out to the screen.

Apart from VRAM, most other types of dual-ported RAM are based on [static RAM](#) technology.

Most CPUs implement the [processor registers](#) as a small dual-ported or multi-ported RAM.



4. XILINX SOFTWARE

4.1 INTRODUCTION

Xilinx ISE^[2] (Integrated Synthesis Environment)^[3] is a software tool produced by [Xilinx](#) for synthesis and analysis of [HDL](#) designs, enabling the developer to [synthesize](#) ("compile") their designs, perform [timing analysis](#), examine [RTL](#) diagrams, simulate a design's reaction to different stimuli, and configure the target device with the [programmer](#).

Xilinx ISE is a design environment for FPGA products from Xilinx, and is tightly-coupled to the architecture of such chips, and cannot be used with FPGA products from other vendors.^[3] The Xilinx ISE is primarily used for circuit synthesis and design, while ISIM or the [ModelSim](#) logic

simulator is used for system-level testing.^{[4][5]} Other components shipped with the Xilinx ISE include the Embedded Development Kit (EDK), a Software Development Kit (SDK) and ChipScope Pro.^[6]

Since 2012, Xilinx ISE has been discontinued in favor of [Vivado Design Suite](#), that serves the same roles as ISE with additional features for [system on a chip](#) development.^{[7][8]} Xilinx released the last version of ISE in October 2013 (version 14.7), and states that "ISE has moved into the sustaining phase of its product life cycle, and there are no more planned ISE releases."

4.2 USER INTERFACE

The primary user interface of the ISE is the Project Navigator, which includes the design hierarchy (Sources), a [source code](#) editor (Workplace), an output console (Transcript), and a processes tree (Processes).^{[3][9]}

The Design hierarchy consists of design files (modules), whose dependencies are interpreted by the ISE and displayed as a [tree structure](#).^[3] For single-chip designs there may be one main module, with other modules included by the main module, similar to the `main()` subroutine in [C++](#) programs.^[3] Design constraints are specified in modules, which include pin configuration and mapping.^[3]

The Processes hierarchy describes the operations that the ISE will perform on the currently active module.^[3] The hierarchy includes compilation functions, their dependency functions, and other utilities.^[3] The window also denotes issues or errors that arise with each function.^[3]

The Transcript window provides status of currently running operations, and informs engineers on design issues.^[3] Such issues may be filtered to show Warnings, Errors, or both.

4.3 SIMULATION

System-level testing may be performed with ISIM or the [ModelSim](#) logic simulator, and such test programs must also be written in HDL languages.^[3] Test bench programs may include simulated input signal waveforms, or monitors which observe and verify the outputs of the [device under test](#).^[3]

[ModelSim](#) or ISIM may be used to perform the following types of simulations:^[4]

- Logical verification, to ensure the module produces expected results
- Behavioural verification, to verify logical and timing issues
- Post-place & route simulation, to verify behaviour after placement of the module within the reconfigurable logic of the FPGA.
 - Xilinx's patented algorithms for synthesis allow designs to run up to 30% faster than competing programs, and allows greater logic density which reduces project time and costs.^[10]
 - Also, due to the increasing complexity of FPGA fabric, including memory blocks and I/O blocks, more complex synthesis algorithms were developed that separate unrelated modules into *slices*, reducing post-placement errors.^[10]
 - IP Cores are offered by Xilinx and other third-party vendors, to implement system-level functions such as [digital signal processing](#) (DSP), bus interfaces, networking protocols, [image processing](#), [embedded processors](#), and peripherals.^[10] Xilinx has been

instrumental in shifting designs from ASIC-based implementation to FPGA-based implementation.

5 FPGA

5.1 INTRODUCTION

A **field-programmable gate array (FPGA)** is an [integrated circuit](#) designed to be configured by a customer or a designer after manufacturing – hence the term "[field-programmable](#)". The FPGA configuration is generally specified using a [hardware description language](#)(HDL), similar to that used for an [application-specific integrated circuit](#) (ASIC). [Circuit diagrams](#) were previously used to specify the configuration, but this is increasingly rare due to the advent of [electronic design automation](#) tools.



A Spartan FPGA from Xilinx

FPGAs contain an array of [programmable logic blocks](#), and a hierarchy of reconfigurable interconnects that allow the blocks to be "wired together", like many logic gates that can be interwired in different configurations. [Logic blocks](#) can be configured to perform complex [combinational functions](#), or merely simple [logic gates](#) like [AND](#) and [XOR](#). In most FPGAs, logic blocks also include [memory elements](#), which may be simple [flip-flops](#) or more complete blocks of memory.^[1] Many FPGAs can be reprogrammed to implement different [logic functions](#),^[2] allowing flexible [reconfigurable computing](#) as performed in [computer software](#).

5.2 TECHNICAL DESIGN

Contemporary field-programmable gate arrays (FPGAs) have large resources of [logic gates](#) and RAM blocks to implement complex digital computations.^[2] As FPGA designs employ very fast I/O rates and bidirectional data [buses](#), it becomes a challenge to verify correct timing of valid data within setup time and hold time.

[Floor planning](#) enables resource allocation within FPGAs to meet these time constraints. FPGAs can be used to implement any logical function that an [ASIC](#) can perform. The ability to update the functionality after shipping, [partial re-configuration](#) of a portion of the design^[3] and the low non-recurring engineering costs relative to an ASIC design (notwithstanding the generally higher unit cost), offer advantages for many applications.^[1]

Some FPGAs have analog features in addition to digital functions. The most common analog feature is a programmable [slew rate](#) on each output pin, allowing the engineer to set low rates on lightly loaded pins that would otherwise [ring](#) or [couple](#) unacceptably, and to set higher rates on heavily loaded pins on high-speed channels that would otherwise run too slowly.^{[4][5]} Also common are quartz-[crystal oscillators](#), on-chip resistance-capacitance oscillators, and [phase-locked loops](#) with embedded [voltage-controlled oscillators](#) used for clock generation and management and for high-speed serializer-deserializer (SERDES) transmit clocks and receiver clock recovery. Fairly common are differential [comparators](#) on input pins designed to be connected to [differential signaling](#) channels. A few "mixed signal FPGAs" have integrated peripheral [analog-to-digital converters](#) (ADCs) and [digital-to-analog converters](#) (DACs) with analog signal conditioning blocks allowing them to operate as a [system-on-a-chip](#) (SoC).^[6] Such devices blur the line between an FPGA, which carries digital ones and zeros on its internal programmable interconnect fabric, and [field-programmable analog array](#) (FPAA), which carries analog values on its internal programmable interconnect fabric.

5.2 HISTORY

The FPGA industry sprouted from [programmable read-only memory](#) (PROM) and [programmable logic devices](#) (PLDs). PROMs and PLDs both had the option of being programmed in batches in a factory or in the field (field-programmable). However, programmable logic was hard-wired between logic gates.^[7]

Altera was founded in 1983 and delivered the industry's first reprogrammable logic device in 1984 – the EP300 – which featured a quartz window in the package that allowed users to shine an ultra-violet lamp on the die to erase the EPROM cells that held the device configuration.^[8] In December 2015, Intel acquired Altera.

Xilinx co-founders Ross Freeman and Bernard Vonderschmitt invented the first commercially viable field-programmable gate array in 1985 – the XC2064.^[9] The XC2064 had programmable gates and programmable interconnects between gates, the beginnings of a new technology and market.^[10] The XC2064 had 64 configurable logic blocks (CLBs), with two three-input lookup tables (LUTs).^[11] More than 20 years later, Freeman was entered into the National Inventors Hall of Fame for his invention.^{[12][13]}

In 1987, the Naval Surface Warfare Center funded an experiment proposed by Steve Casselman to develop a computer that would implement 600,000 reprogrammable gates. Casselman was successful and a patent related to the system was issued in 1992.^[7]

Altera and Xilinx continued unchallenged and quickly grew from 1985 to the mid-1990s, when competitors sprouted up, eroding significant market share. By 1993, Actel (now Microsemi) was serving about 18 percent of the market.^[10] By 2013, Altera (31 percent), Actel (10 percent) and Xilinx (36 percent) together represented approximately 77 percent of the FPGA market.^[14]

The 1990s were a period of rapid growth for FPGAs, both in circuit sophistication and the volume of production. In the early 1990s, FPGAs were primarily used in telecommunications and networking. By the end of the decade, FPGAs found their way into consumer, automotive, and industrial applications.

5.3 APPLICATION

An FPGA can be used to solve any problem which is computable. This is trivially proven by the fact that FPGAs can be used to implement a soft microprocessor, such as the Xilinx MicroBlaze or Altera Nios II. Their advantage lies in that they are significantly faster for some applications because of their parallel nature and optimality in terms of the number of gates used for certain processes.^[34]

FPGAs originally began as competitors to CPLDs to implement glue logic for printed circuit boards. As their size, capabilities, and speed increased, FPGAs took over additional functions to the point where some are now marketed as full systems on chips (SoCs). Particularly with the introduction of dedicated multipliers into FPGA architectures in the late 1990s, applications which had traditionally been the sole reserve of digital signal processor hardware (DSPs) began to incorporate FPGAs instead.^{[35][36]}

Another trend in the use of FPGAs is hardware acceleration, where one can use the FPGA to accelerate certain parts of an algorithm and share part of the computation between the FPGA and a generic processor.^[2] The search engine Bing is noted for adopting FPGA acceleration for its search algorithm in 2014.^[37] As of 2018, FPGAs are seeing increased use as AI accelerators including Microsoft's so-termed "Project Catapult"^[19] and for accelerating artificial neural networks for machine learning applications.

Traditionally,^[when?] FPGAs have been reserved for specific vertical applications where the volume of production is small. For these low-volume applications, the premium that companies pay in hardware cost per unit for a programmable chip is more affordable than the development

resources spent on creating an ASIC. As of 2017, new cost and performance dynamics have broadened the range of viable applications.

In Italy, the company [Biomine srl](#) has patented the application of FPGA processors, improving its yield in crypto currency mining.

6.Mimas V2 Spartan 6 FPGA Development Board with DDR SDRAM

6.1 INTRODUCTION

MIMAS V2 is a feature packed yet low cost FPGA Development board featuring Xilinx Spartan-6 FPGA. It is specially designed for experimenting and learning system design with FPGAs. This development board features SPARTAN XC6SLX9 CSG324 FPGA with onboard 512Mb DDR SDRAM. The USB 2.0 interface provides fast and easy configuration download to the on-board SPI flash. No need to buy an expensive programmer or special downloader cable to download the bit stream to the board.

Applications

- Product Prototype Development
- Signal Processing
- Learning Digital Electronics
- Educational tool for schools and universities

Board features

- FPGA: Spartan XC6SLX9 in CSG324 package
- DDR: 166MHz 512Mb LPDDR (MT46H32M16LF/W949D6CBHX6E)
- Flash memory: 16 Mb SPI flash memory (M25P16)
- USB 2.0 interface for On-board flash programming
- FPGA configuration via JTAG and USB
- 8 LEDs Six Push Buttons and 8 way DIP switch for user defined purposes
- VGA Connector
- Stereo Jack
- Micro SD Card Adapter
- Three Digit Seven Segment Display.
- 32 IOs for user defined purposes
- Four 6×2 Expansion Connectors

- On-board voltage regulators for single power rail operation

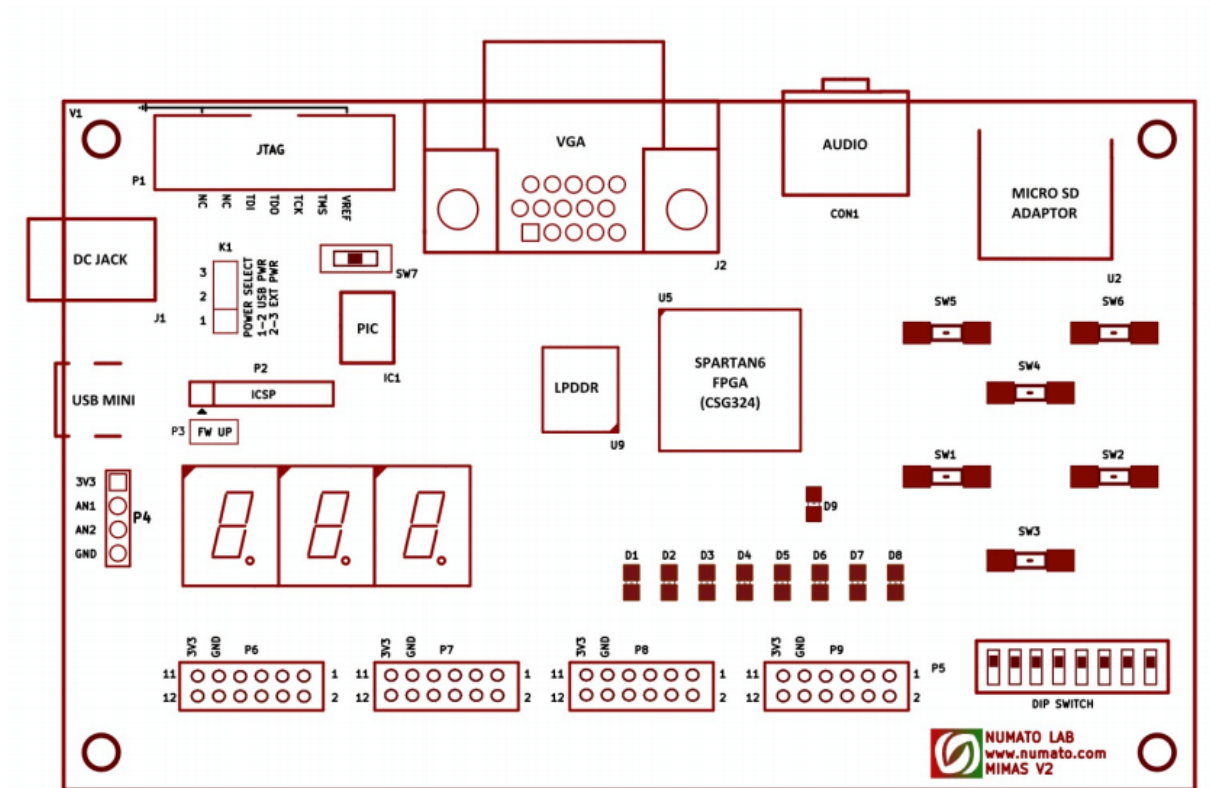
5.2 HOW TO USE MIMAS SPARTANV2 FPGA BOARD

5.2.1 COMPONENTS REQUIRED

Along with the module, you may need the items in the list below for easy and fast installation.

1. USB A to Mini B cable.
2. DC Power supply (Optional).

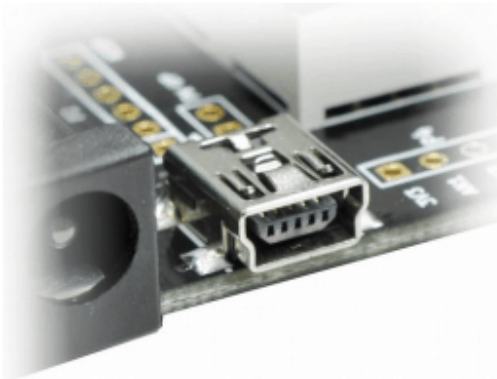
5.2.2 CONNECTION DIAGRAM



This diagram should be used as a reference only. For detailed information, see MIMAS V2 schematics at the end of this documentation. Details of individual connectors are as below.

5.3 USB INTERFACE

The on board full speed USB controller helps a computer to communicate with this module. Use a USB A to Mini B cable to connect with a PC. By default the module is powered from USB so make sure not to overcrowd unpowered USB hubs.



5.4 DC POWER SUPPLY

This module uses +5V power supply to function properly. **By default the board is configured to use +5V supply from USB. So an external +5V power is not required unless USB port is unable to supply enough current. In most cases USB ports are capable of providing enough current for the module. Current requirement for this board largely depends on your application. Please consult FPGA datasheet for more details on power requirements.** If for any reason, an external 5V power supply needs to be used for the module, the Power select jumper should be configured properly before connecting the power supply. Please refer to the marking on the board for more details.

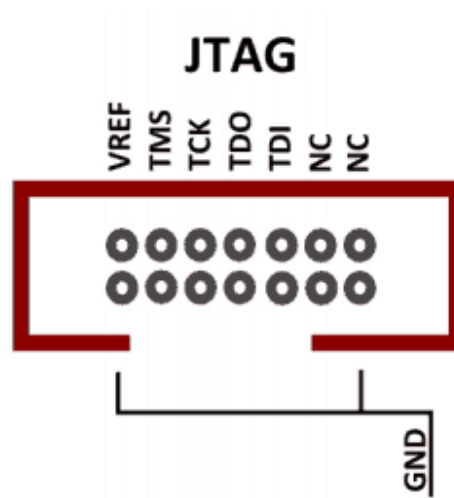


5.5 POWER SELECTOR

The Power Select header K1 is used to configure the power source for the board. The jumper in pin 1 and 2 is shorted to switch the power source to on board USB port and pin 2 and 3 to use the external DC power.

5.6 JTAG CONNECTOR

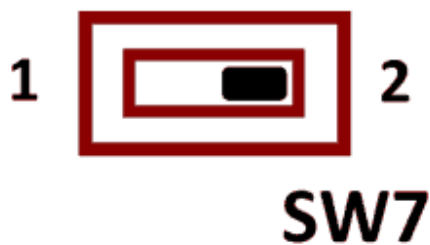
JTAG connector provides access to FPGA's JTAG pins. A XILINX platform cable can be



used for JTAG programming.

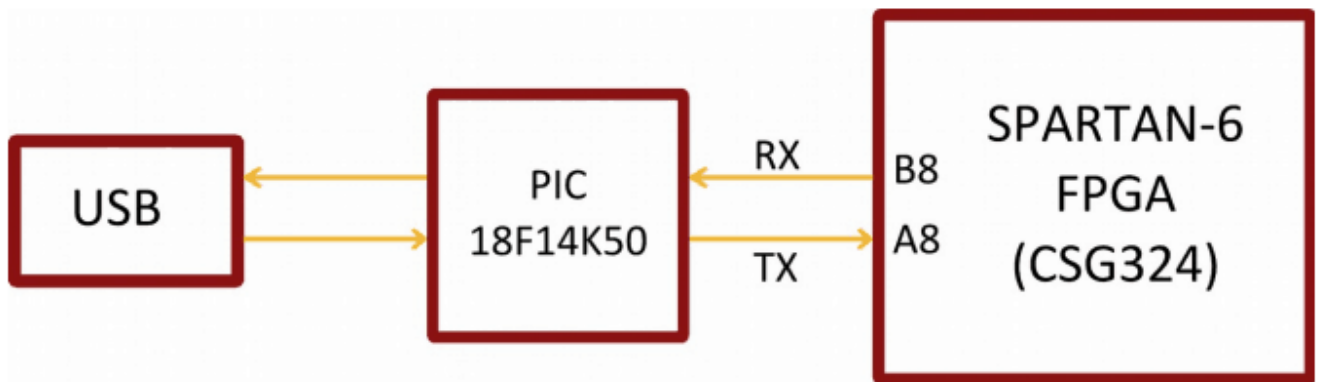
5.7 CONFIGURATION MODE SELECTION

Slide switch SW7 is used to switch between the USB configuration mode and UART. Slide the switch to Position 1 to download bit stream through USB configuration tool and Position 2 to use the interface as a UART in order to communicate from your code in FPGA with the PC. By default the board is shipped with slide switch position in USB configuration tool mode.



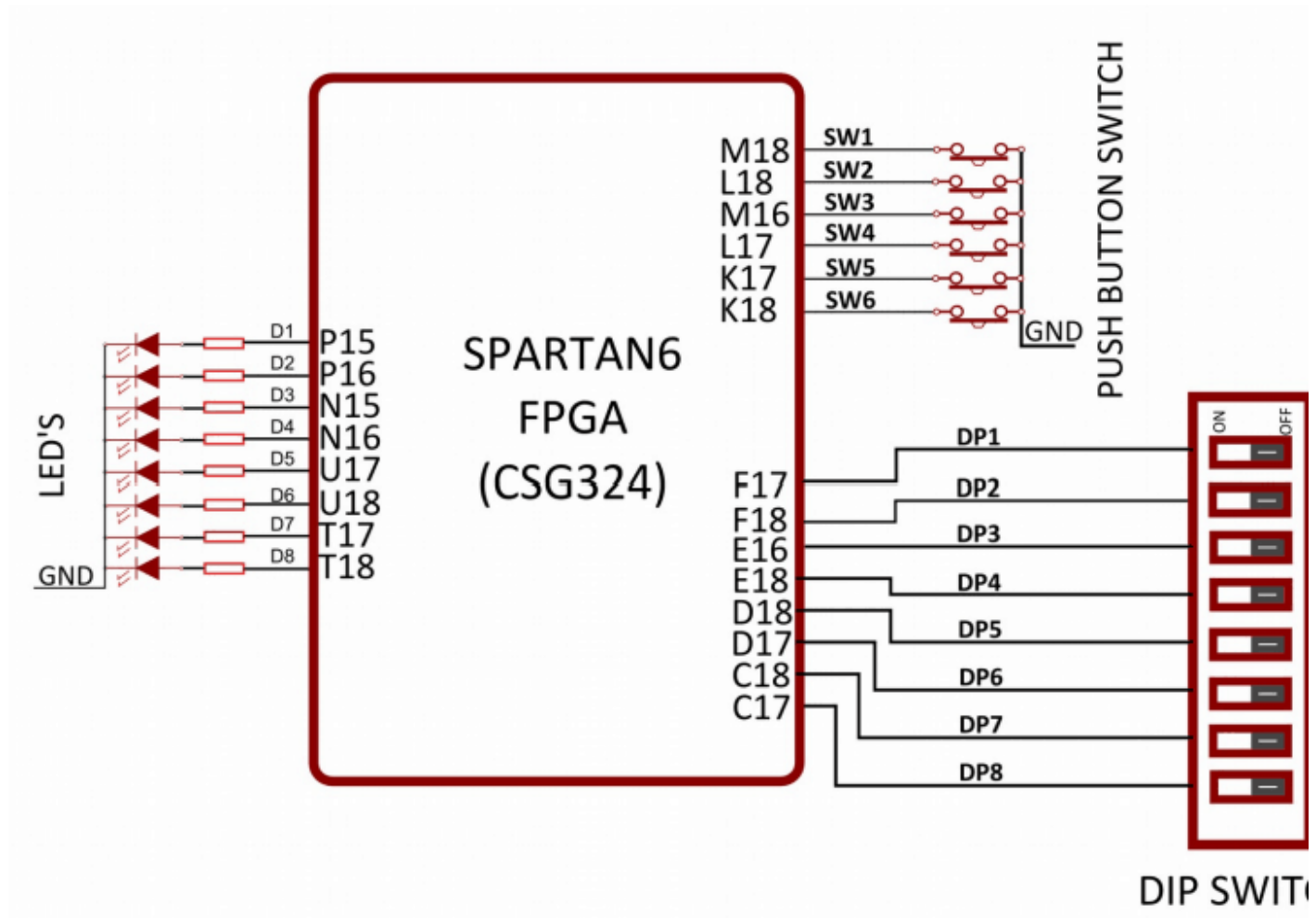
5.8 UART

The MIMAS V2 includes USB-UART, which helps to establish the communication between the code in the FPGA and any application running on the PC. Data can be sent and received from the FPGA by using Serial Terminal at baud rate 19200.



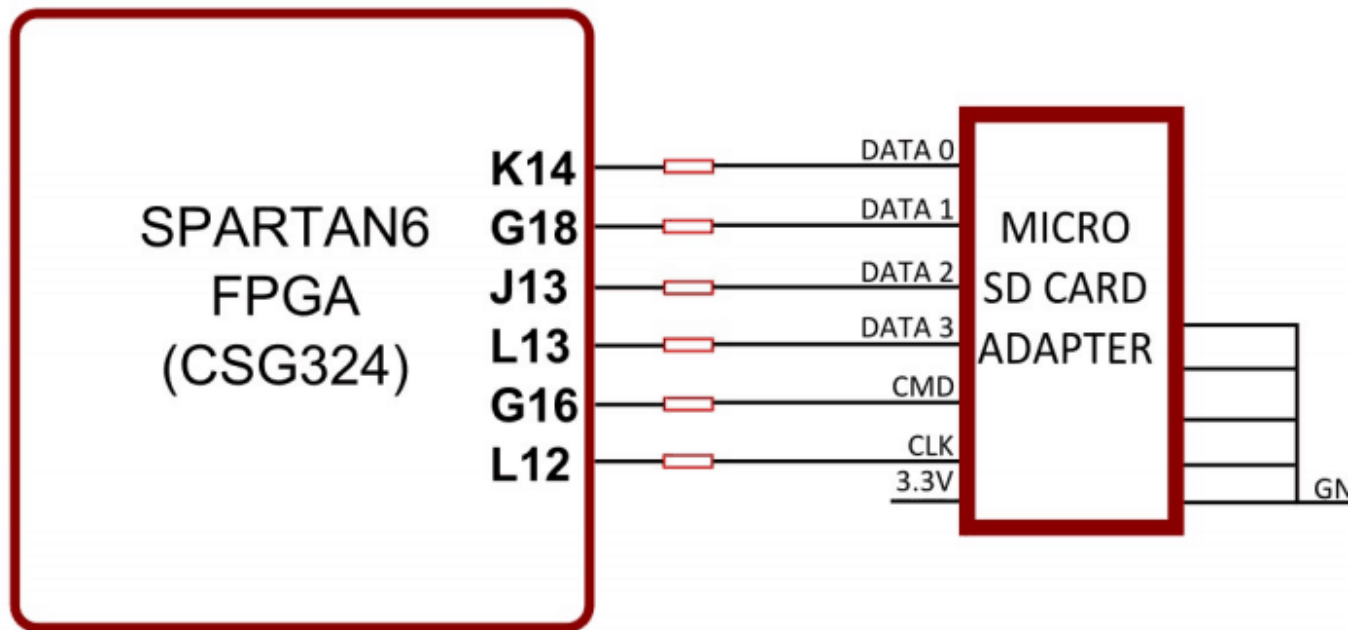
5.9 LED, Push Button and Dip Switch

MIMAS V2 has six push button switches, an eight position DIP switch and eight LEDs for human interaction. All switches are directly connected to Spartan 6 FPGA and can be used in your design with minimal effort.



5.10 MICRO SD

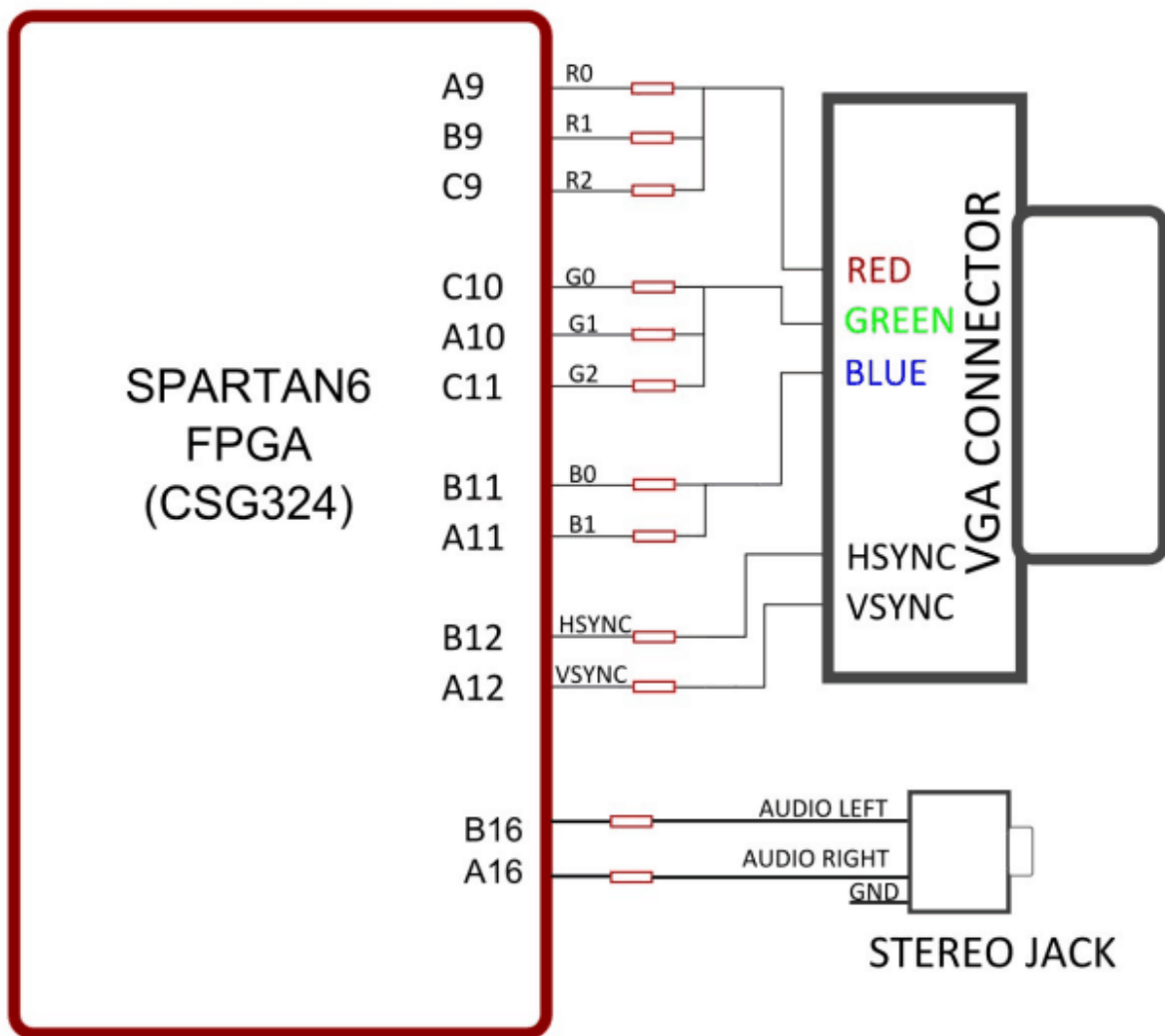
MIMAS V2 features a Micro SD adapter on-board. By installing a Micro SD card, you can add data logging, media storage and other file storage to your design.



5.11 VGA AND AUDIO

The VGA interface provides this board the ability to generate VGA signals from FPGA and display information any Display/monitor that supports standard VGA connector. This VGA interface uses resistor network based DAC for easy code implementation. This 8 bit VGA interface can display up to 256 colors.

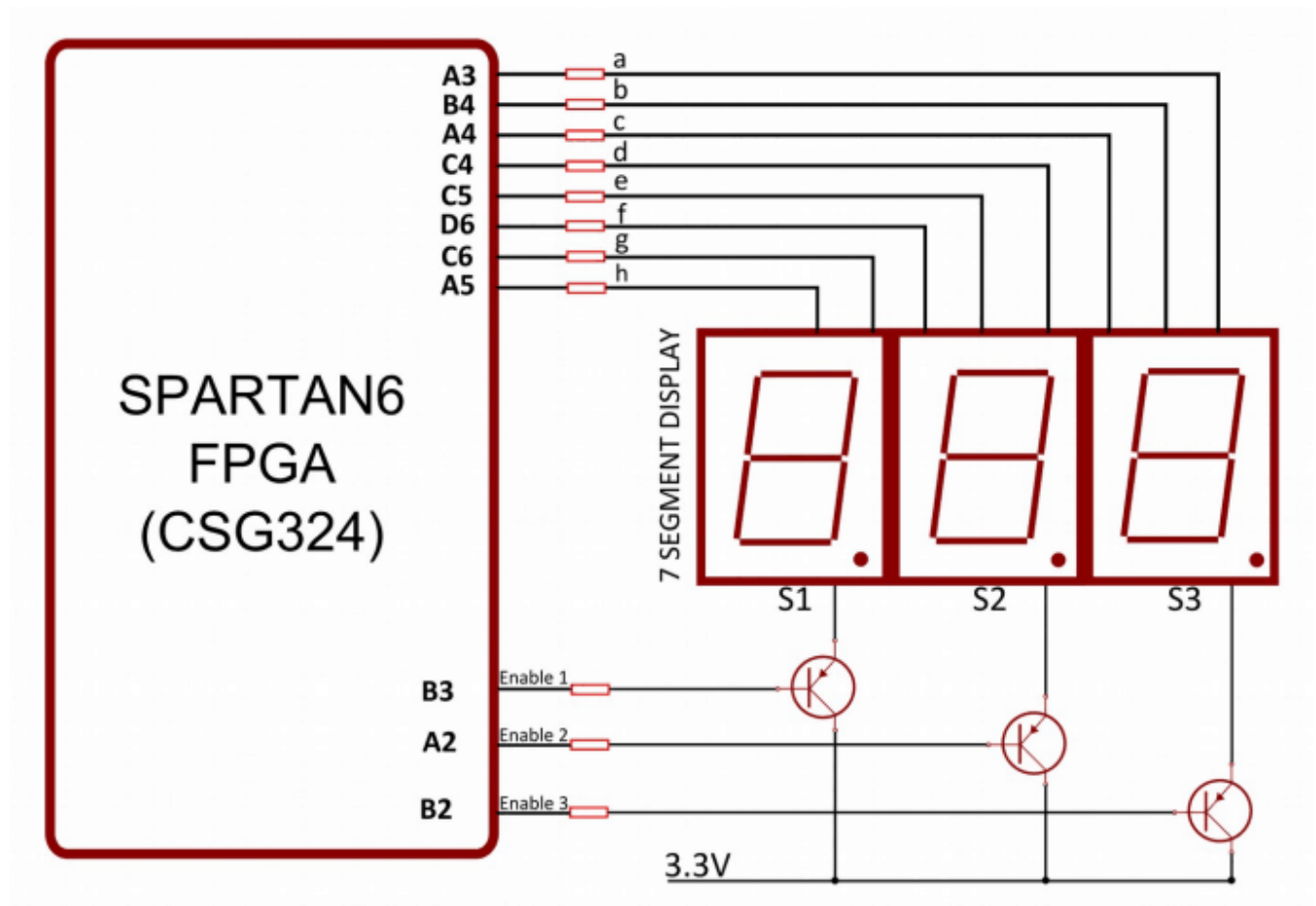
Two IOs on the FPGA are dedicated for generating two channels of audio. Different audio tones can be generated by using PWM and Frequency synthesis.



5.13 SEGMENT LED DISPLAY

This board features three 7-segment LED display multiplexed for low pin count operation. Each module can be separately turned on and off with the three switching transistors.

Note: All signals (*a, b, c, d, e, f, g, dot, enable 1, enable 2, enable 3*) used for controlling 7-Segment display are **active-low** signals. So, for example, for displaying “8” in display-2, users need to drive *Enable 2* to 0 as well as drive signals *a, b, c, d, e, f* to 0. All other signals need to be driven to 1.



5.12 GPIOs

This board is equipped with 32 user IO pins that can be used for various custom applications. Pin assignments on the connectors are available in the tables below.

HEADER P6

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
1	IO_L43P_2	U7
2	IO_L43N_2	V7

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
3	IO_L63P_2	T4
4	IO_L63N_2	V4
5	IO_L49P_D3_2	U5
6	IO_L49N_D4_2	V5
7	IO_L62P_D5_2	R3
8	IO_L62N_D6_2	T3
9	GND	NA
10	GND	NA
11	VCCAUX	NA

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
12	VCCAUX	NA

HEADER P7

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
1	IO_L41P_2	U8
2	IO_L41N_VREF_2	V8
3	IO_L31P_GCLK31_D14_2	R8
4	IO_L31N_GCLK30_D15_2	T8
5	IO_L48P_D7_2	R5
6	IO_L48N_RDWR_B_VREF_2	T5
7	IO_L32P_GCLK29_2	T9

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
8	IO_L32N_GCLK28_2	V9
9	GND	NA
10	GND	NA
11	VCCAUX	NA
12	VCCAUX	NA

HEADER P8

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
1	IO_L16P_2	R11
2	IO_L16N_VREF_2	T11
3	IO_L29P_GCLK3_2	R10

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
4	IO_L29N_GCLK2_2	T10
5	IO_L14P_D11_2	U13
6	IO_L14N_D12_2	V13
7	IO_L23P_2	U11
8	IO_L23N_2	V11
9	GND	NA
10	GND	NA
11	VCCAUX	NA
12	VCCAUX	NA

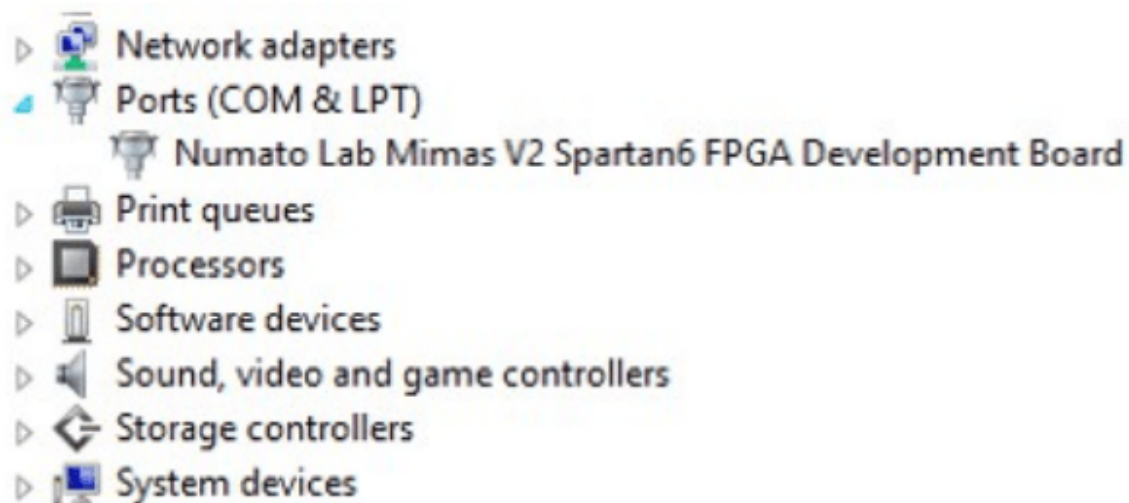
HEADER P9

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
1	IO_L43P_GCLK5_M1DQ4_1	H17
2	IO_L43N_GCLK4_M1DQ5_1	H18
3	IO_L44P_A3_M1DQ6_1	J16
4	IO_L44N_A2_M1DQ7_1	J18
5	IO_L41P_GCLK9_IRDY1_M1RASN_1	K15
6	IO_L41N_GCLK8_M1CASN_1	K16
7	IO_L42P_GCLK7_M1UDM_1	L15
8	IO_L42N_GCLK6_TRDY1_M1LDM_1	L16
9	GND	NA

Header Pin No.	Pin description	Spartan-6 (CSG324) Pin No.
10	GND	NA
11	VCCAUX	NA
12	VCCAUX	NA

6.Driver Installation

6.1.Installing on Windows



This product requires a driver to be installed for proper functioning when used with Windows. The driver package can be downloaded from the product page. To install the driver, unzip the contents of the downloaded driver package to a folder. Attach USB cable to the PC and when asked by Windows device installation wizard, point to the folder where driver files are present. When driver installation is complete, the module should appear in

Windows Device Manager as a serial port (see the picture on the right). Note down the name of the serial port (COM1, COM2 etc..). This information is required while programming the module with configuration tool.

6.2 Installing on Linux

To use this product with Linux, USB CDC driver needs to be compiled in with the kernel. Fortunately, **most Linux distributions (Ubuntu, Redhat, Debian etc.) has this driver pre-installed**. The chances of you requiring to rebuild the kernel to include the USB CDC driver is very slim. When connected to a Linux machine, this product should appear as a serial port in the /dev directory. Usually the name of the device will be “ttyACMx” or similar. The name may be different depending on the Linux distribution you have.

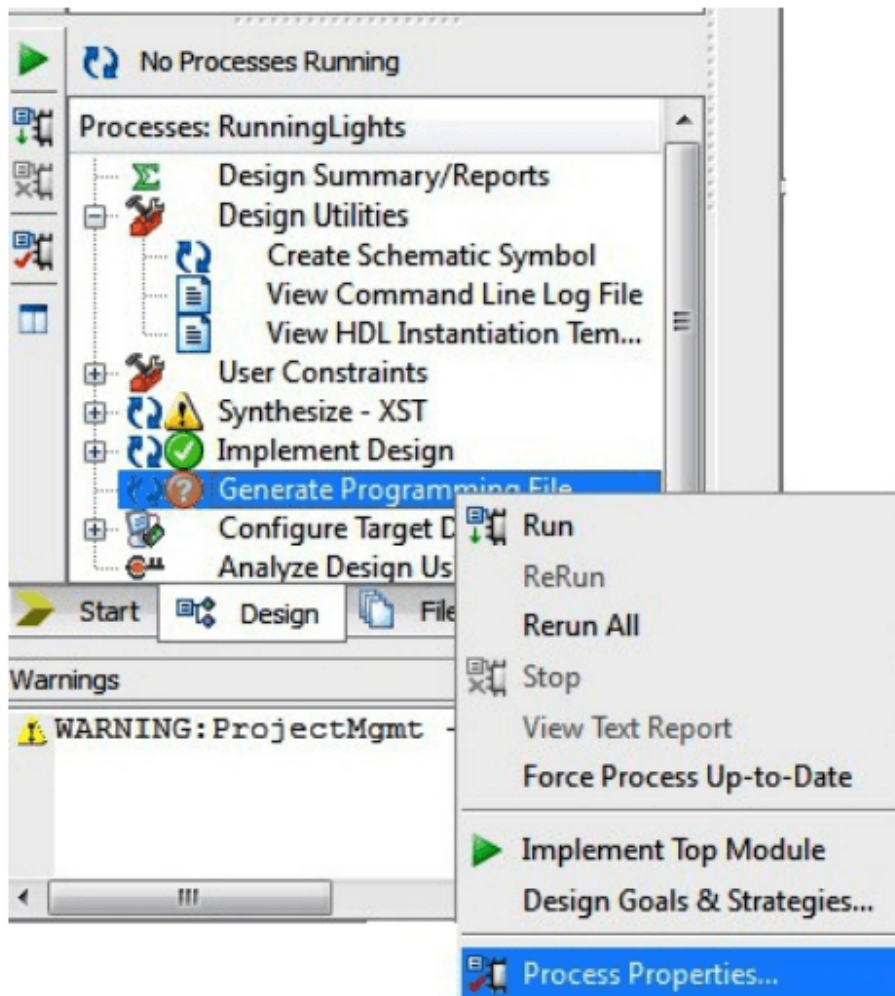
6.3 Installing on Mac

Similar to Linux, Mac operating system comes with the required drivers pre-installed. When connected to a Mac computer, the device should appear as a serial port.

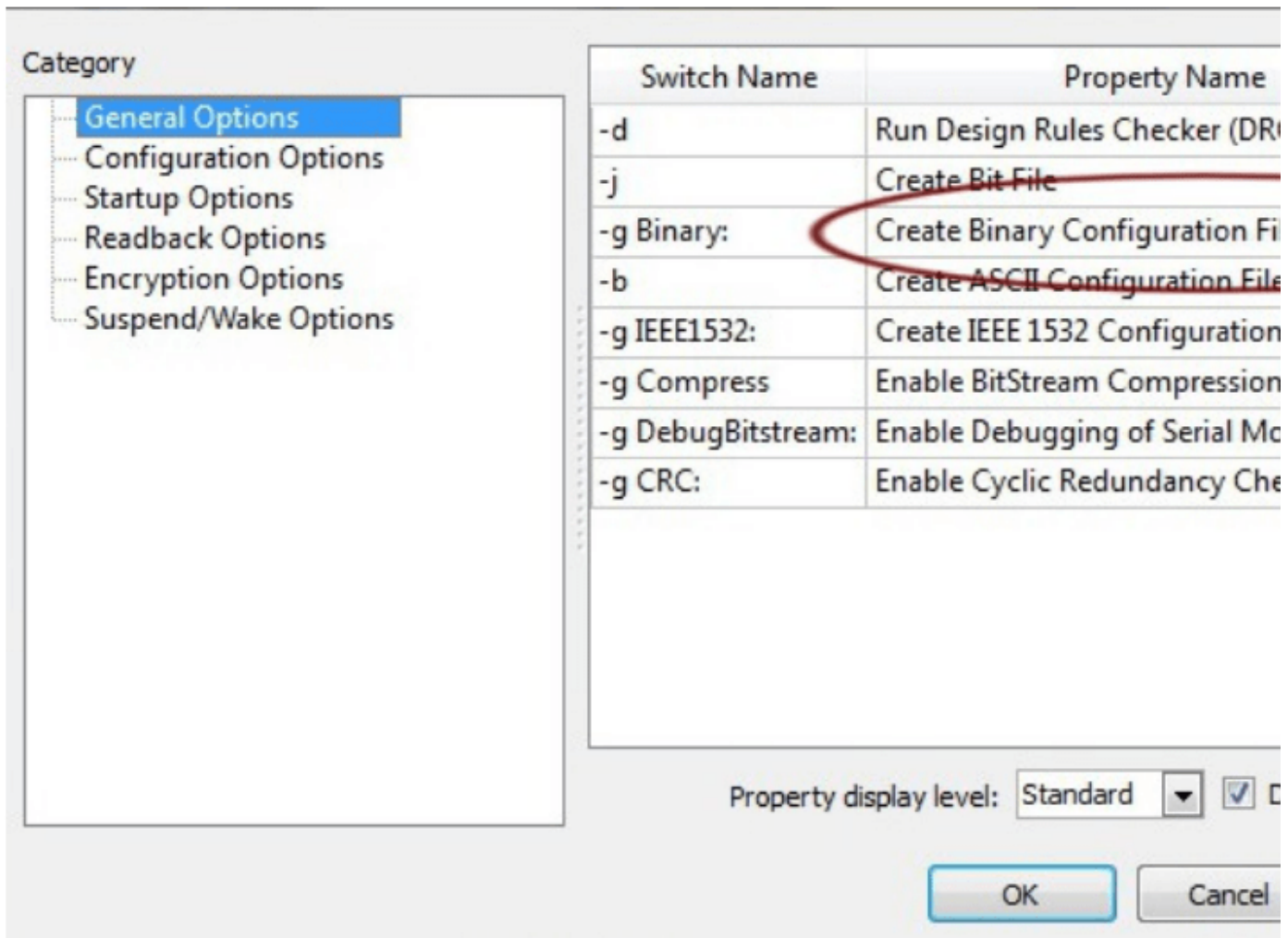
6.4 Generating Bit Stream for MIMAS V2

HDL design needs to be converted to bit stream before it can be programmed to FPGA. MIMAS V2 configuration tool at this time accepts only binary (.bin) bit stream created by XILINX ISE (<http://www.xilinx.com/tools/webpack.htm>). Once the HDL is synthesized, it is easy to create a binary bit stream out of it. Please follow the Steps below to generate binary bit stream from your design using ISE Web Pack.

Step 1: Right click on the “Generate Programming File” option in “Processes” window.



Step 2: Select “Process Properties” from the pop up menu. In the dialog box, check “Create Binary Configuration File” Check box and click “Apply”.



Step 3: Click “OK” to close the dialog box. Right click on “Generate Programming File” option again and select “Run”. Now you will be able to find a .bin file in the project directory and that file can be used for MIMAS V2 configuration.

6.5 Powering Up MIMAS V2

MIMAS V2 can be powered directly from USB port so make sure that you are using a USB port that can power the board properly. It is recommended to connect the board directly to the PC instead using a hub. It is practically very difficult to estimate the power consumption of the board, as it depends heavily on your design and the clock used. XILINX provides tools to estimate the power consumption. In any case if power from USB is not enough for your application, an external supply can be applied to the board. MIMAS V2 requires two

different voltages, a 3.3V and a 1.2V supply. On-board regulators derive these voltages from the USB/Ext power supply

6.6 Configuring MIMAS V2

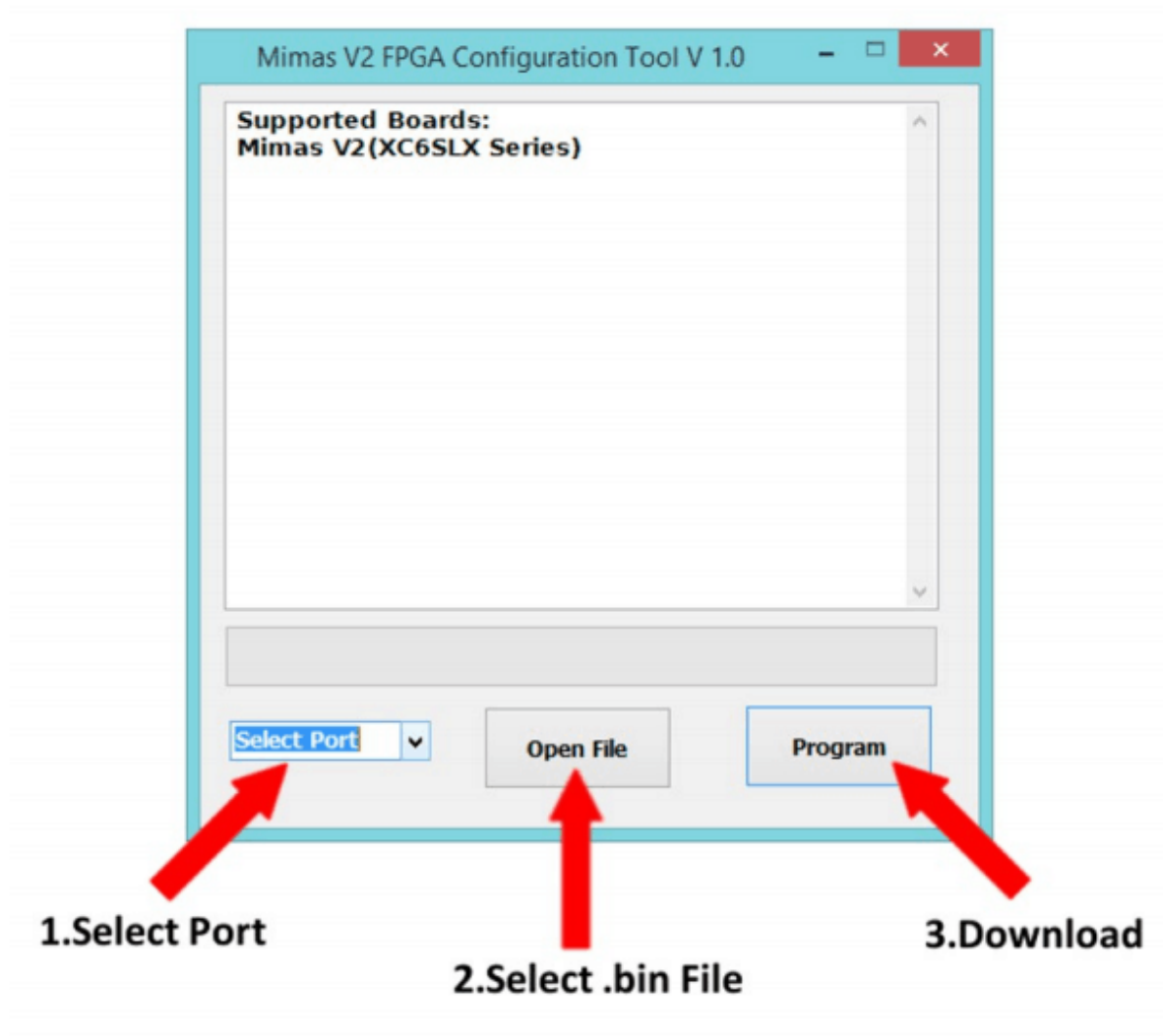
The MIMAS V2 Spartan6 module can be configured by two methods,

- a) Using MIMAS V2 configuration tool through USB.
- b) Using the Xilinx programming cable.

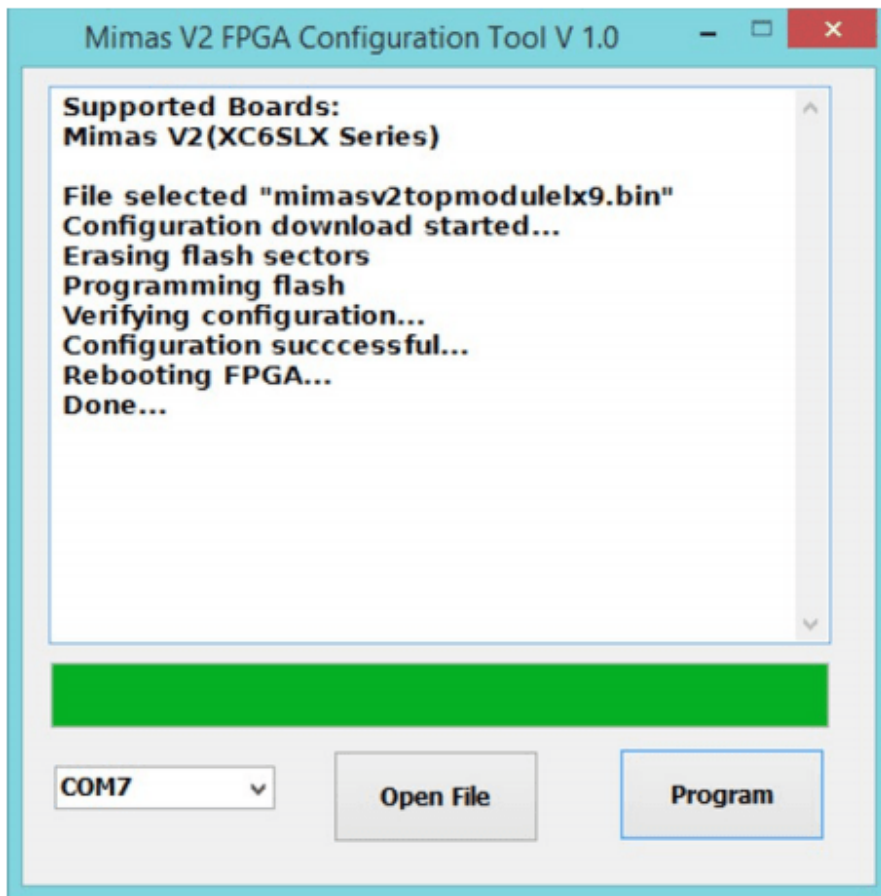
6.7 Configuring MIMAS V2 Using Configuration Tool

MIMAS V2 has an on-board micro-controller which facilitates easy reprogramming of on-board SPI flash through USB interface. The micro-controller receives bit stream from the host application and program it in to the SPI Flash and lets the FPGA boot from the flash. The MIMAS V2 configuration application can be downloaded from www.numato.com for free. When MIMAS V2 is connected to PC, it shows up as a COM port in Device Manager. Run configuration application, select correct COM Port before downloading bit stream. Click on “Open File” to select the bit stream file (.bin) and press “Program” button to download the bit stream. Wait till the download process is finished. Once the download process is over, the configuration controller will try to boot the FPGA from the SPI Flash automatically. Follow the below steps.

Step 1: Make sure you have selected USB configuration mode (Slide SW7 to position 1. Refer to the section “Configuration Mode Selection” for more information). Run MIMAS V2 Configuration Tool and select the correct port (Refer to section “Driver installation” for more information on finding port number). Click Open file button and select the .bin file.



Step 2: Click on “Program” button. Wait till “Done” appears on the screen.

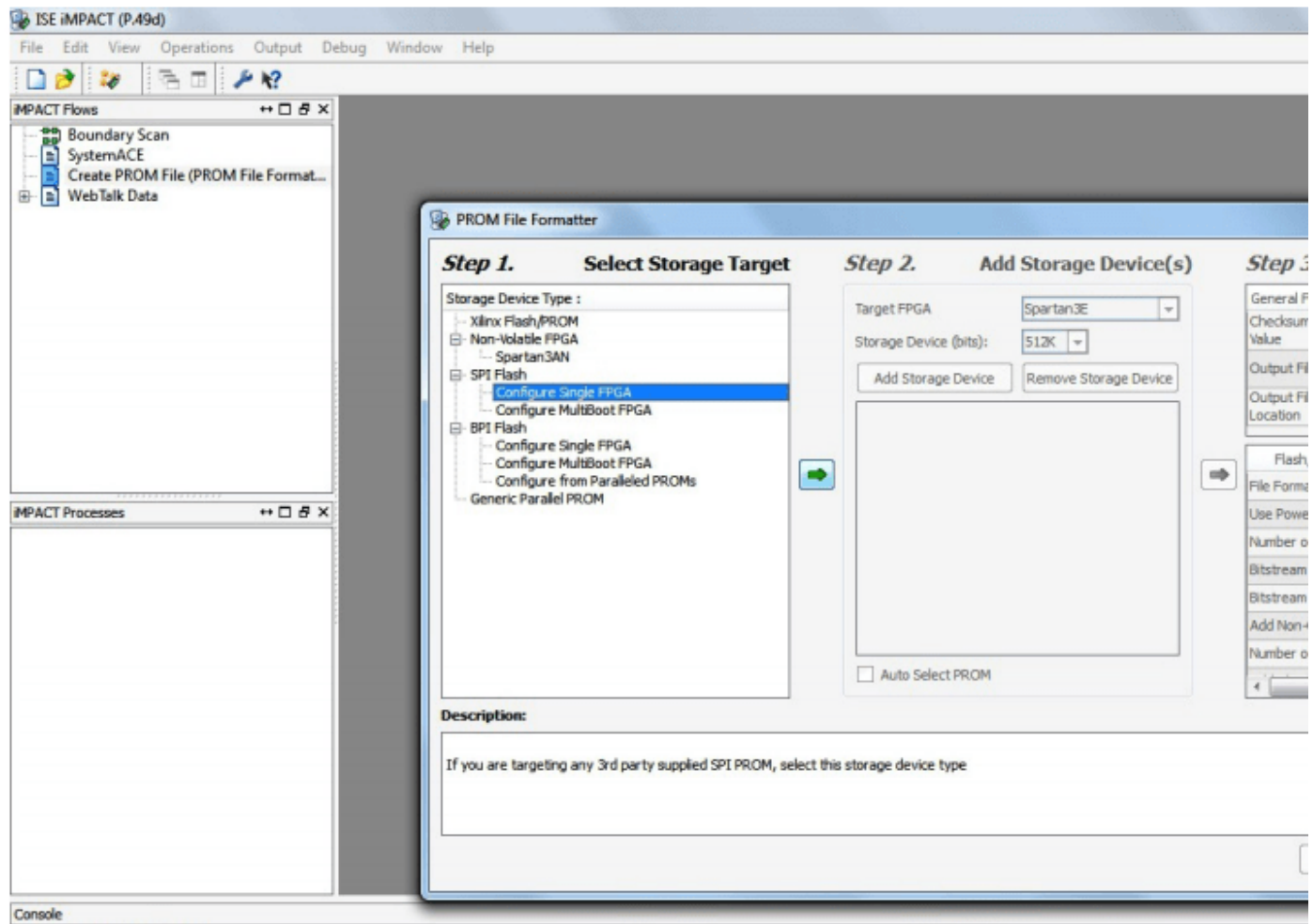


6.8 Configuring MIMAS V2 Using JTAG

MIMAS V2 Spartan6 module features an on-board JTAG connector which facilitates easy reprogramming of SRAM and on-board SPI flash through JTAG programmer like “XILINX Platform-cable usb”. Programming MIMAS V2 using JTAG requires “XILINX ISE iMPACT” software which is bundled with XILINX ISE Design Suite. To program SPI flash we need a “.mcs” file which needs to be generated from the “.bit” file. Steps for generating “.mcs” file is discussed below. Programming FPGA SRAM does not require a mcs file to be generated.

Generating “.mcs” file for MIMAS V2

Step 1: Open ISE iMPACT. Click on “Create PROM file(PROM file formatter)”. In the dialog box, select “Configure Single FPGA” in storage device type. Then click on the green arrow at the right side.



Step 2: Select 16M in Storage Device (bits) list. Now click on “Add Storage Device”, then the green arrow at the right side.

Step 2. Add Storage Device(s)

Storage Device (bits)

Add Storage Device

128K
256K
512K
1M
2M
4M
8M
16M
32M
64M
128M
256M

Auto Select PROM

Step 3: Set an output file name and an output file location (the “.mcs” file will be generated at this location which will be required later for programming the FPGA), then click OK twice, then select the “.bit” file we already generated then click Open and click NO when it prompts to add another device file.

Step 3.
Enter Data

General File Detail	Value
Checksum Fill Value	FF
Output File Name	MimasV2
Output File Location	C:\Xilinx\14.4\ISE_DS\ 



Flash/PROM File Property	Value
File Format	MCS 
Add Non-Configuration Data Files	No 

d mode.

ed to calculate the checksum of the unused portions.

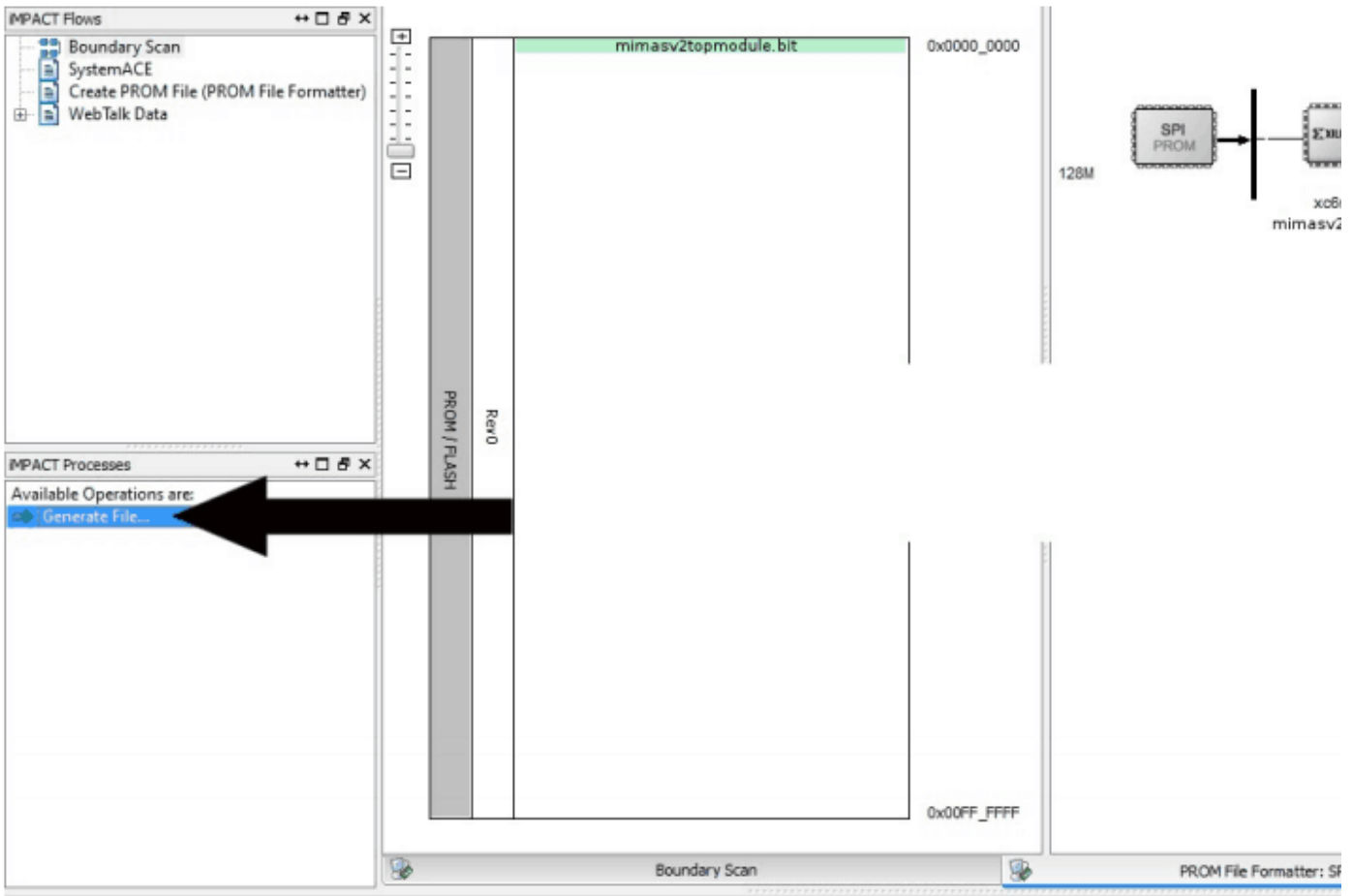
le format your PROM programmer uses, you output a MCS

OK

Cancel

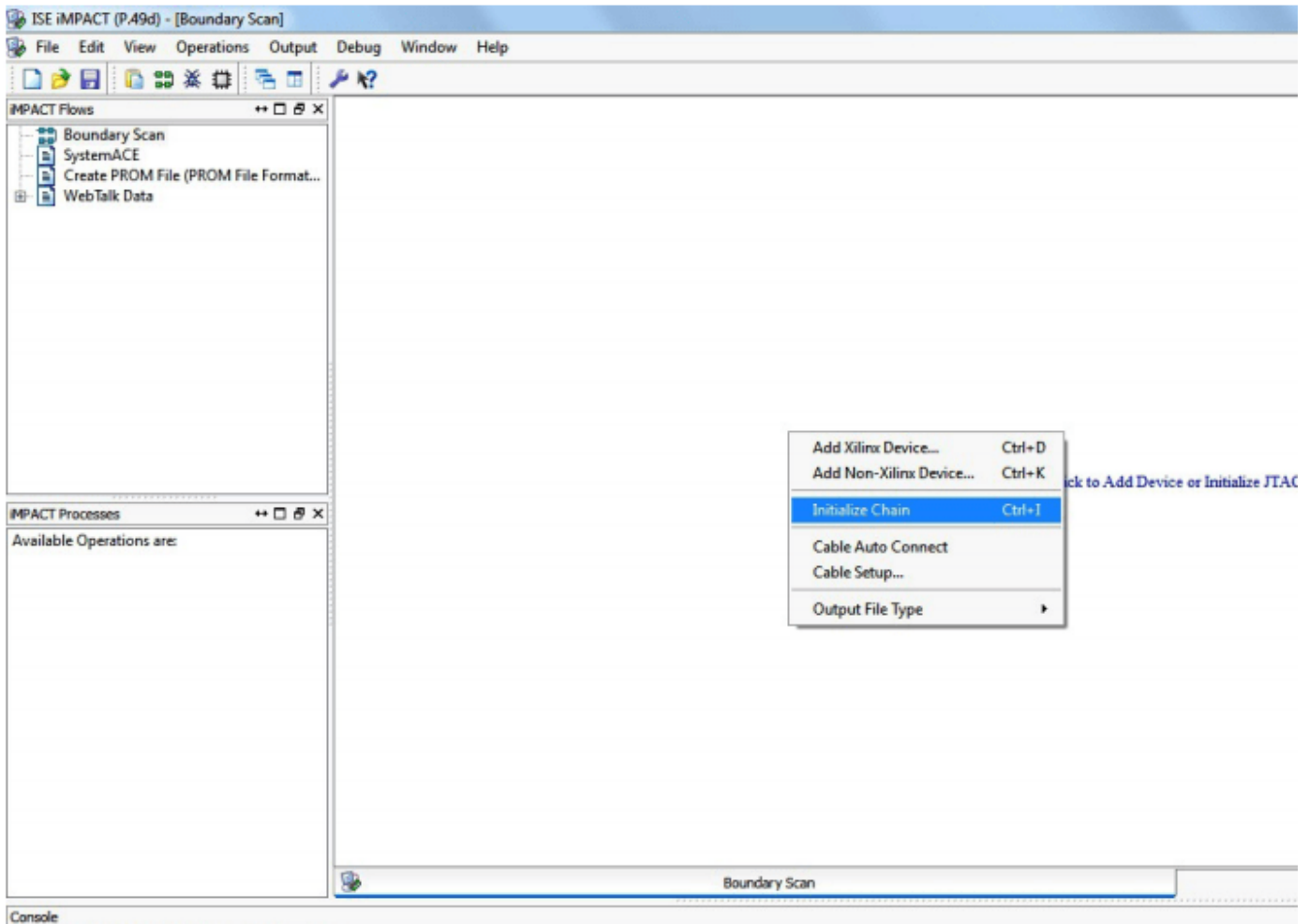
Help

Step 4: Double click on “Generate File”. A message “Generate Succeeded” will be displayed as shown in fig below if the mcs file is generated successfully.

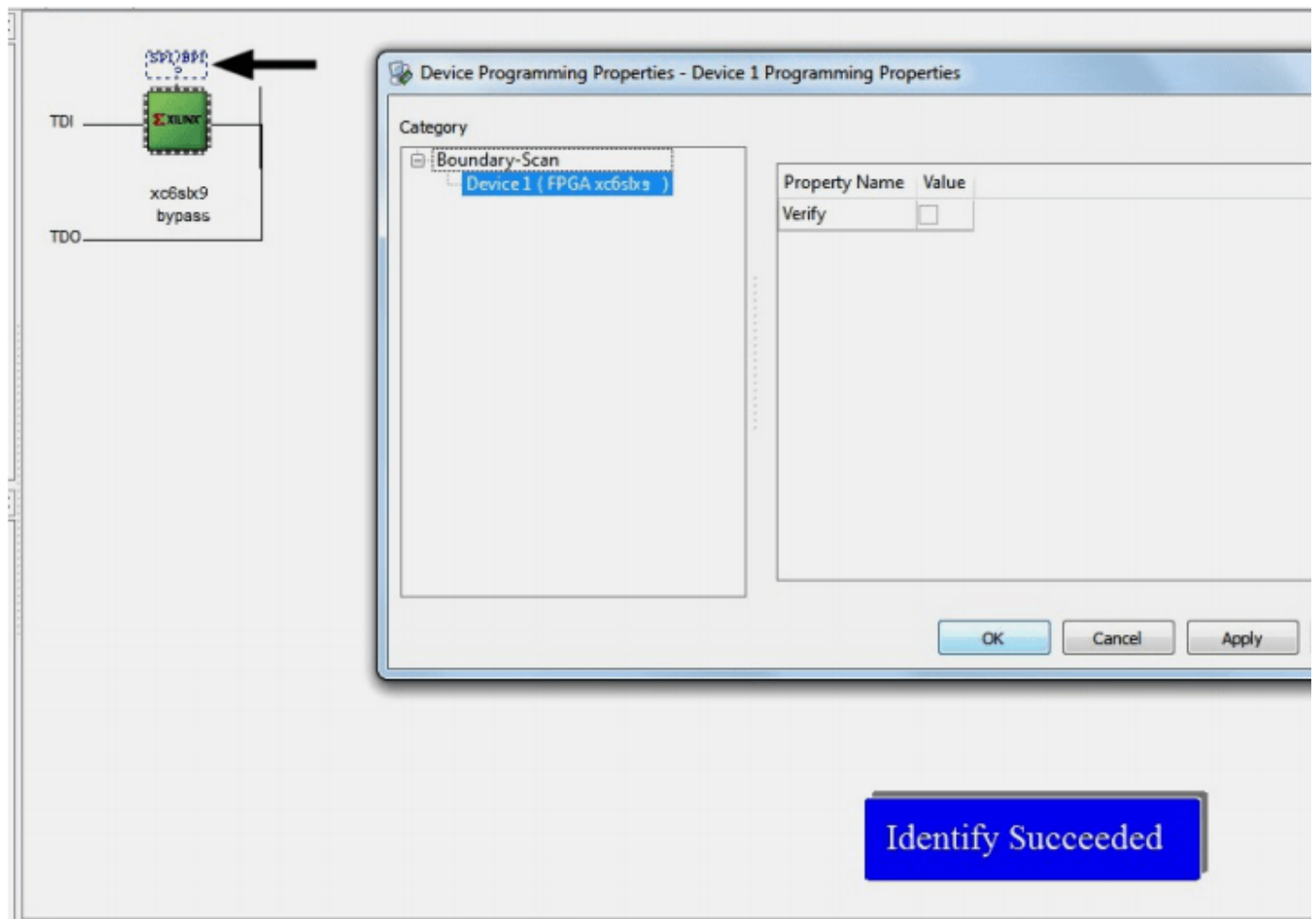


6.9 Programming Onboard SPI flash Using ISE iMPACT

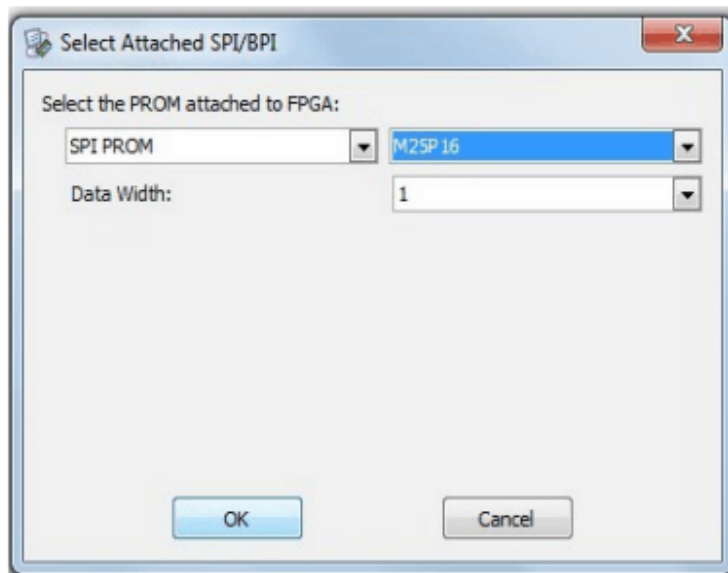
Step 1: Make sure Xilinx Platform Cable USB is connected properly to the board. Open ISE iMPACT. Click on “Boundary Scan” in the iMPACT flows window in the left top corner. Then right click on the window panel in the right side. Select “Initialize Chain”.



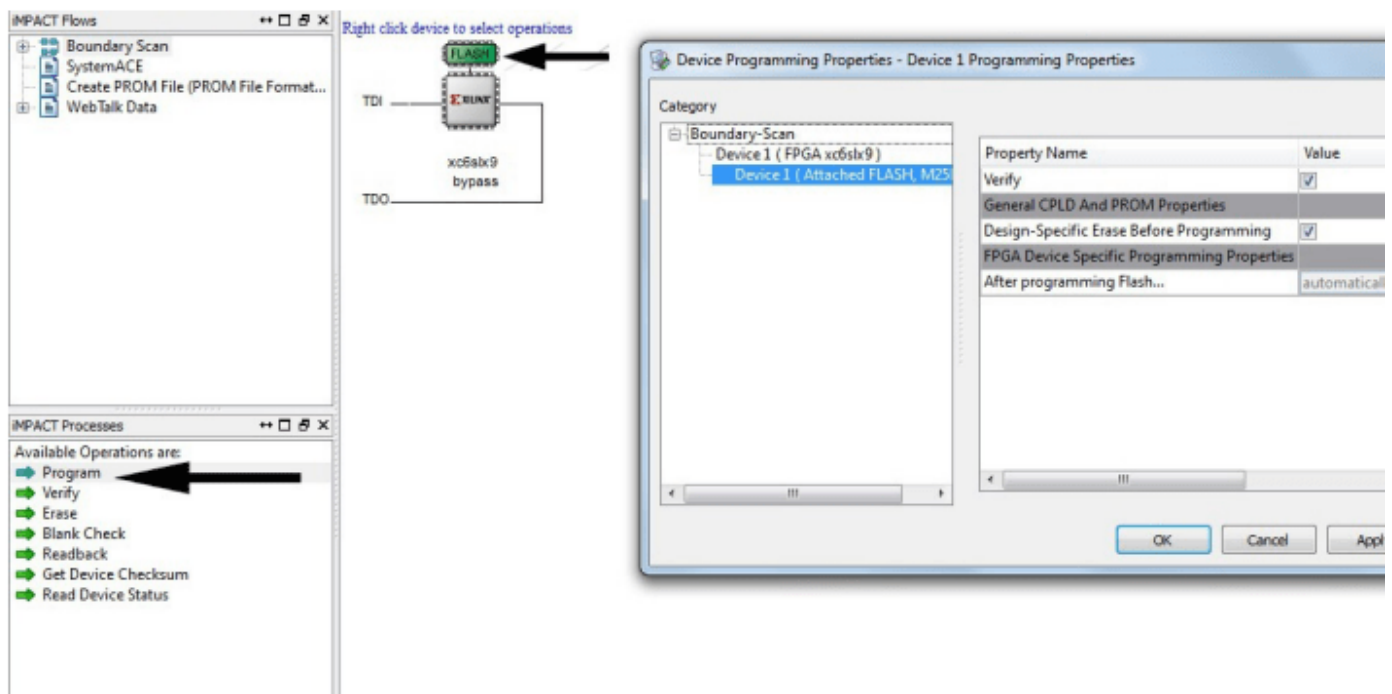
Step 2: If the device is detected properly you will get a pop up window as shown below, Click OK. Then right click on the SPI/BPI (next to the black arrow in the below fig.), select Add SPI/BPI Flash.



Step 3: Select the “.mcs” file we already created and click OK. Now choose “M25P16” in the dialogue box appeared, then click OK.



Step 4: Click on “Flash”, Double Click on Program, select OK. If the programming is successful, a confirmation message will be displayed.



5. CODING

CAM CODING

```
library ieee;
```

```
    use ieee.std_logic_1164.all;
```

```
    use ieee.std_logic_unsigned.all;
```

```
    use ieee.std_logic_arith.all;
```

```
entity cam is
```

```
    generic (
```

```
        ADDR_WIDTH :integer := 8
```

```
    );
```

```
    port (
```

```
        clk      :in std_logic;
```

```
        cam_enable :in std_logic;
```

```
        cam_data_in :in std_logic_vector (2**ADDR_WIDTH-1 downto 0);
```

```
        cam_hit_out :out std_logic;
```

```
        cam_addr_out:out std_logic_vector (ADDR_WIDTH-1 downto 0)
```

```
    );
```

```
end entity;
```

```
architecture rtl of cam is
```

```
    constant DEPTH :integer := 2**ADDR_WIDTH;
```

```
    signal cam_addr_combo :std_logic_vector (ADDR_WIDTH-1 downto 0);
```

```

signal cam_hit_combo :std_logic;

signal found_match :std_logic;

begin

process (cam_data_in, cam_hit_combo, cam_addr_combo, found_match) begin

    cam_addr_combo <= (others=>'0');

    found_match <= '0';

    cam_hit_combo <= '0';

    for i in 0 to DEPTH-1 loop

        if (cam_data_in(i) = '1' and found_match = '0') then

            found_match <= '1';

            cam_hit_combo <= '1';

            cam_addr_combo <= conv_std_logic_vector(i, cam_addr_combo'length);

        else

            found_match <= found_match;

            cam_hit_combo <= cam_hit_combo;

            cam_addr_combo <= cam_addr_combo;

        end if;

    end loop;

end process;

process (clk) begin

```

```

if (rising_edge(clk)) then
    if (cam_enable = '1') then
        cam_hit_out <= cam_hit_combo;
        cam_addr_out <= cam_addr_combo;
    else
        cam_hit_out <= '0';
        cam_addr_out <= (others=>'0');
    end if;
end if;
end process;

```

```

end architecture;

```

DUAL PORT SSRAM CODING

```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.STD_LOGIC_UNSIGNED.ALL;

```

```

entity dual_port_ram is
    port(
        clk: in std_logic;

        wr_en : in std_logic;

        data_in : in std_logic_vector(7 downto 0);

        addr_in_0 : in std_logic_vector(3 downto 0);

```



```

    addr_in_1 : in std_logic_vector(3 downto 0);

    port_en_0 : in std_logic; --enable port 0.

    port_en_1 : in std_logic; --enable port 1.

    data_out_0 : out std_logic_vector(7 downto 0);

    data_out_1 : out std_logic_vector(7 downto 0)

);

end dual_port_ram;

```

architecture Behavioral of dual_port_ram is

```

type ram_type is array(0 to 15) of std_logic_vector(7 downto 0);

signal ram : ram_type := (others => (others => '0'));

```

```

begin

```

```

    process(clk)

```

```

    begin

```

```

        if(rising_edge(clk)) then

```

```

            if(port_en_0 = '1') then

```

```

                if(wr_en = '1') then

```

```

                    ram(conv_integer(addr_in_0)) <= data_in;

```

```

                end if;

```

```
        end if;  
    end if;  
end process;
```

```
data_out_0 <= ram(conv_integer(addr_in_0)) when (port_en_0 = '1') else  
    (others => 'Z');  
data_out_1 <= ram(conv_integer(addr_in_1)) when (port_en_1 = '1') else  
    (others => 'Z');  
  
end Behavioral;
```

TEST BENCH CODE FOR DUALPORT SRAM

```
LIBRARY ieee;  
USE ieee.std_logic_1164.ALL;  
use IEEE.STD_LOGIC_ARITH.ALL;  
  
ENTITY tb IS  
  
END tb;
```

ARCHITECTURE behavior OF tb IS

COMPONENT dual_port_ram

PORT(

clk : IN std_logic;

wr_en : IN std_logic;

data_in : IN std_logic_vector(7 downto 0);

addr_in_0 : IN std_logic_vector(3 downto 0);

addr_in_1 : IN std_logic_vector(3 downto 0);

port_en_0 : IN std_logic;

port_en_1 : IN std_logic;

data_out_0 : OUT std_logic_vector(7 downto 0);

data_out_1 : OUT std_logic_vector(7 downto 0)

);

END COMPONENT;

signal clk : std_logic := '0';

signal wr_en : std_logic := '0';

signal data_in : std_logic_vector(7 downto 0) := (others => '0');

signal addr_in_0 : std_logic_vector(3 downto 0) := (others => '0');

```
signal addr_in_1 : std_logic_vector(3 downto 0) := (others => '0');
```

```
signal port_en_0 : std_logic := '0';
```

```
signal port_en_1 : std_logic := '0';
```

```
signal data_out_0 : std_logic_vector(7 downto 0);
```

```
signal data_out_1 : std_logic_vector(7 downto 0);
```

```
constant clk_period : time := 10 ns;
```

```
BEGIN
```

```
uut: dual_port_ram PORT MAP (
```

```
    clk => clk,
```

```
    wr_en => wr_en,
```

```
    data_in => data_in,
```

```
    addr_in_0 => addr_in_0,
```

```
    addr_in_1 => addr_in_1,
```

```
    port_en_0 => port_en_0,
```

```
    port_en_1 => port_en_1,
```

```
    data_out_0 => data_out_0,
```

```
    data_out_1 => data_out_1
```

```
);
```

```
clk_process :process
begin
    clk <= '1';
    wait for clk_period/2;
    clk <= '0';
    wait for clk_period/2;
end process;
```

```
stim_proc: process
begin

    port_en_0 <= '0';
    wr_en <= '1';
    data_in <= X"FF";
    addr_in_0 <= X"1";
    wait for 20 ns;

    port_en_0 <= '1';
    for i in 1 to 16 loop
        data_in <= conv_std_logic_vector(i,8);
```

```

        addr_in_0 <= conv_std_logic_vector(i-1,4);

        wait for 10 ns;

    end loop;

    wr_en <= '0';

    port_en_0 <= '0';

    port_en_1 <= '1';

    for i in 1 to 16 loop

        addr_in_1 <= conv_std_logic_vector(i-1,4);

        wait for 10 ns;

    end loop;

    port_en_1 <= '0';

    wait;

end process;

END;

```

6. OUPUT

2.2. Thermal Summary

Thermal Summary		
Effective TJA (C/W)	30.5	
Max Ambient (C)	84.6	
Junction Temp (C)	25.4	

2.3. Power Supply Summary

Power Supply Summary			
	Total	Dynamic	Static Power
Supply Power (mW)	14.11	0.45	13.66